App Contatos com Auth — Expo + TypeScript + Expo Router (API v2 com GridFS)

> Versão didática e super comentada. Pensada para sala de aula: cada etapa explica o **porquê**, o **como** e os **erros comuns** que seus alunos vão encontrar.

✅ Resumo geral do aplicativo

Este aplicativo é um **gerenciador de contatos com autenticação** desenvolvido em **Expo (React Native + TypeScript + Expo Router)** e conectado a uma **API Node.js/MongoDB com GridFS**.

Fluxo principal de uso

1. O usuário **cria uma conta** (nome, e-mail, senha) e faz **login**.
2. Ao entrar, recebe um **token JWT** que é salvo no aparelho (com **SecureStore**) e usado em todas as requisições.
3. Com o token válido, ele acessa a área de **contatos**:
   - **Listar** contatos existentes (com foto, se houver).
   - **Criar** um novo contato preenchendo nome, telefone, e-mail, endereço e opcionalmente enviando uma foto.
   - **Editar** um contato já existente (dados e foto).
   - **Excluir** contatos.
4. Para fotos, o app envia a imagem via `POST /upload` e recebe um `fileId`. Esse `fileId` é salvo no contato e usado depois em `GET /upload/:id` para exibir a imagem.

O que o aluno verá funcionando

- **Autenticação segura**: apenas usuários logados acessam contatos.
- **Persistência**: o login não se perde ao fechar o app, pois o token é salvo de forma segura com **SecureStore**.
- **CRUD completo de contatos**: adicionar, listar, editar e excluir.
- **Upload e exibição de fotos** com integração ao GridFS.

> Resumindo: é um app prático que mostra todo o ciclo **Autenticação → CRUD → Upload de imagem**.

---

1) Pré-requisitos (e como explicar para a turma)

- **Node.js LTS ≥ 18** e **Expo** instalados.
- A **API v2** rodando (local ou servidor) com estas rotas:

- `POST /usuarios/registrar`, `POST /usuarios/login` (JWT)
- `GET/POST/PUT/DELETE /contatos`
- `POST /upload` (multipart campo `foto`) → `{ fileId }`
- `GET /upload/:id` → stream da imagem

- **URL base da API** anotada (ex.: `http://192.168.0.10:3000`).

Por que isso importa?

Porque o app vai **precisar do token** (JWT) para acessar as rotas de contatos e **precisa conhecer a URL correta** para falar com o backend.

Erros comuns

- Aluno tenta usar `http://localhost:3000` no celular físico. **No celular,** `localhost` **é o próprio celular**, não o PC. Use o **IP da máquina**.
- No Android **emulador** (AVD), usar `10.0.2.2:3000` em vez de `localhost`.
- No iOS **simulador**, `http://localhost:3000` funciona.

Dica: deixe um slide com uma tabela "URL por plataforma" (trazemos um exemplo mais abaixo, na Configuração de `BASE_URL`).

---

2) Criar o projeto (Expo + TS + Router)

```
npx create-expo-app app-contatos --template
cd app-contatos
```

Instale as dependências usadas neste guia:

```
npx expo install axios expo-image-picker expo-secure-store react-native-web
react-dom

npx expo install expo-router
```

Por que essas libs?

- **axios**: fácil de centralizar headers e interceptores (token).
- **expo-image-picker**: seleciona/captura imagens com permissões já integradas.
- **expo-secure-store**: guarda o **JWT** de forma criptografada, mais seguro que AsyncStorage.

Teste rápido: Rode `npx expo start` e confirme que o app "Hello world" abre no **Web**, **Android** e **iOS** (se disponível).

Migrar Blank TS → Expo Router

1. **Excluir** `App.tsx` **da raiz**

- O `App.tsx` não é mais usado, porque o roteamento passa a ser controlado pela pasta `app/`.

2. **Editar** `index.ts`
    - Substitua todo o conteúdo por:

```
import "expo-router/entry";
```

- Isso registra automaticamente o roteador como ponto de entrada, eliminando a necessidade do `registerRootComponent`.

3. **Criar a pasta** `app/`
    - Cada arquivo dentro dela vira uma rota.

---

3) Estrutura sugerida do projeto

```
app-contatos/
├─ app/
│  ├─ _layout.tsx            # Layout com guarda de autenticação
│  ├─ index.tsx              # Login
│  ├─ cadastro.tsx           # Cadastro de usuário
│  └─ contatos/
│     ├─ index.tsx           # Lista de contatos
│     ├─ novo.tsx            # Novo contato
│     └─ [id].tsx            # Editar contato
├─ components/
│  └─ FormContato.tsx        # Formulário reutilizável (com upload e preview)
├─ lib/
│  └─ api.ts                 # Axios + Token + helpers (usando SecureStore)
├─ styles/
│  └─ global.ts              # Estilos básicos
├─ types/
│  └─ Contato.ts             # Tipagem do contato (fotoId)
└─ assets/
   └─ placeholder.png        # Imagem opcional quando não há foto
```

Por que assim? Mantém telas simples e separa **infra** (axios/token), **UI reutilizável** (form) e **tipos**. Alunos entendem melhor quando a responsabilidade de cada arquivo é clara.

---

4) Tipos — `types/Contato.ts`

```
export interface Contato {
  _id: string;
  nome: string;
  email?: string;
  telefone?: string;
  endereco?: string;
  /** ID do arquivo no GridFS retornado por POST /upload */
  fotoId?: string;
}
```

Por que `fotoId?: string`**?** Porque o contato **pode** ter sido criado sem foto, e depois receber uma.

---

5) Armazenamento do token com SecureStore

Agora usamos a biblioteca **expo-secure-store** para guardar o JWT com mais segurança.

`lib/api.ts`

```
import axios from 'axios';
import * as SecureStore from 'expo-secure-store';

const BASE_URL = 'http://SEU_BACKEND:3000'; // <- troque para a sua API

const api = axios.create({ baseURL: BASE_URL });

export async function setAuthToken(token: string) {
  await SecureStore.setItemAsync('token', token, { keychainAccessible:
SecureStore.AFTER_FIRST_UNLOCK });
  api.defaults.headers.common['Authorization'] = `Bearer ${token}`;
}

export async function loadAuthToken() {
  const token = await SecureStore.getItemAsync('token');
  if (token) api.defaults.headers.common['Authorization'] = `Bearer ${token}`;
  return token;
}

export async function clearAuthToken() {
  await SecureStore.deleteItemAsync('token');
  delete api.defaults.headers.common['Authorization'];
}

export function getImageUrl(fileId?: string) {
  return fileId ? `${BASE_URL}/upload/${fileId}` : undefined;
}

export default api;
```

Explicação : O SecureStore garante que o token esteja protegido pelo sistema operacional (Keychain no iOS, Keystore no Android). Isso é mais seguro que AsyncStorage, que grava em texto simples.

---

6) Estilos — `styles/global.ts`

```
import { StyleSheet } from 'react-native';

export const global = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#fff' },
  input: { borderWidth: 1, borderColor: '#ccc', padding: 10, marginBottom: 10,
borderRadius: 6 },
  button: { padding: 12, backgroundColor: '#0d6efd', borderRadius: 6,
alignItems: 'center', marginVertical: 10 },
  buttonText: { color: '#fff', fontWeight: 'bold' },
  title: { fontSize: 18, fontWeight: 'bold', marginBottom: 10 }
});
```

Por que centralizar estilos? Para a turma focar na **lógica** do app sem se perder em CSS.

---

7) Configurar BASE_URL corretamente (evita erros)

Temos três estratégias. Escolha **uma** e mantenha igual em todo o curso.

A) Valor fixo (mais simples)

```
// lib/api.ts (apenas recorte da BASE_URL)
const BASE_URL = 'http://192.168.0.10:3000'; // troque pelo IP do seu PC na LAN
```

B) Por plataforma

```
import { Platform } from 'react-native';
const BASE_URL = Platform.select({
  ios: 'http://localhost:3000',      // iOS simulador
  web: 'http://localhost:3000',      // Web
  android: 'http://10.0.2.2:3000',   // Android emulador (AVD)
}) as string;
```

C) Via `extra` no `app.config.ts` (para dev/hml/prod)

```
// app.config.ts
export default ({ config }) => ({
  ...config,
  extra: { API_URL: process.env.API_URL ?? 'http://192.168.0.10:3000' }
});

// lib/api.ts
import Constants from 'expo-constants';
const BASE_URL = (Constants.expoConfig?.extra as any)?.API_URL as string;
```

Tabela mental de URLs

- **Android emulador**: `http://10.0.2.2:3000`
- **iOS simulador**: `http://localhost:3000`
- **Celular físico**: `http://IP_DO_PC:3000`

---

8) Guarda de autenticação com Expo Router — `app/_layout.tsx`

Bloqueia `/contatos/*` para quem não está logado.

```
import { Stack, usePathname, useRouter } from 'expo-router';
import { useEffect, useState } from 'react';
import { ActivityIndicator, View } from 'react-native';
import { loadAuthToken } from '../lib/api';

export default function RootLayout() {
  const [loading, setLoading] = useState(true);
  const pathname = usePathname();
  const router = useRouter();
```

```
  useEffect(() => {
    (async () => {
      const token = await loadAuthToken();
      const isAuthPage = pathname === '/' || pathname === '/cadastro';
      if (!token && !isAuthPage) router.replace('/');
      setLoading(false);
    })();
  }, [pathname]);

  if (loading) {
    return (
      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
        <ActivityIndicator size="large" />
      </View>
    );
  }

  return <Stack screenOptions={{ headerShown: true }} />;
}
```

Como explicar: Antes de carregar as telas, verificamos o token. Sem token, redirecionamos para login.

---

9) Login — `app/index.tsx`

```
import { useState } from 'react';
import { View, TextInput, Button, Alert } from 'react-native';
import { useRouter } from 'expo-router';
import api, { setAuthToken } from '../lib/api';
import { global } from '../styles/global';

export default function Login() {
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');
  const router = useRouter();

  const entrar = async () => {
    try {
      if (!email || !senha) {
        Alert.alert('Campos obrigatórios', 'Preencha e-mail e senha.');
        return;
      }
      const { data } = await api.post('/usuarios/login', { email, senha });
      if (!data?.token) throw new Error('Token não retornado');
      await setAuthToken(data.token);
      router.push('/contatos');
    } catch (err: any) {
      const msg = err?.response?.data?.mensagem || err?.message || 'Falha ao
autenticar.';
      Alert.alert('Erro ao entrar', msg);
    }
  };

  return (
    <View style={global.container}>
      <TextInput placeholder="Email" value={email} onChangeText={setEmail}
style={global.input} autoCapitalize="none" />
      <TextInput placeholder="Senha" value={senha} onChangeText={setSenha}
secureTextEntry style={global.input} />
      <Button title="Entrar" onPress={entrar} />
```

```
      <Button title="Cadastrar" onPress={() => router.push('/cadastro')} />
    </View>
  );
}
```

---

10) Cadastro — `app/cadastro.tsx`

```
import { useState } from 'react';
import { View, TextInput, Button, Alert } from 'react-native';
import { useRouter } from 'expo-router';
import api from '../lib/api';
import { global } from '../styles/global';

export default function Cadastro() {
  const [nome, setNome] = useState('');
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');
  const router = useRouter();

  const registrar = async () => {
    try {
      if (!nome || !email || !senha) {
        Alert.alert('Campos obrigatórios', 'Preencha nome, e-mail e senha.');
        return;
      }
      await api.post('/usuarios/registrar', { nome, email, senha });
      Alert.alert('Sucesso', 'Usuário cadastrado!');
      router.replace('/');
    } catch (err: any) {
      const msg = err?.response?.data?.mensagem || 'Tente novamente.';
      Alert.alert('Erro ao cadastrar', msg);
    }
  };

  return (
    <View style={global.container}>
      <TextInput placeholder="Nome" value={nome} onChangeText={setNome}
style={global.input} />
      <TextInput placeholder="Email" value={email} onChangeText={setEmail}
style={global.input} autoCapitalize="none" />
      <TextInput placeholder="Senha" value={senha} onChangeText={setSenha}
secureTextEntry style={global.input} />
      <Button title="Registrar" onPress={registrar} />
    </View>
  );
}
```

---

11) Lista — `app/contatos/index.tsx`

```
import {  useState } from 'react';
import { View, Text, Button, FlatList, Image, TouchableOpacity, Alert } from
'react-native';
import { useRouter, useFocusEffect } from 'expo-router';
import api, { loadAuthToken, getImageUrl } from '../../lib/api';
import { global } from '../../styles/global';
import { Contato } from '../../types/Contato';

export default function ListaContatos() {
```

```
  const [contatos, setContatos] = useState<Contato[]>([]);
  const router = useRouter();

  const carregar = async () => {
    try {
      await loadAuthToken();
      const { data } = await api.get('/contatos');
      setContatos(data);
    } catch (err) {
      Alert.alert('Erro', 'Falha ao carregar contatos. Verifique login e API.');
    }
  };

  const excluir = async (id: string) => {
    try {
      await api.delete(`/contatos/${id}`);
      carregar();
    } catch {
      Alert.alert('Erro', 'Não foi possível excluir.');
    }
  };

  useFocusEffect(() => {

    carregar();

  });

  return (
    <View style={global.container}>
      <Text style={global.title}>Seus contatos</Text>
      <Button title="Novo contato" onPress={() => router.push('/contatos/novo')}
/>
      <FlatList
        data={contatos}
        keyExtractor={(item) => item._id}
        renderItem={({ item }) => (
          <TouchableOpacity onPress={() =>
router.push(`/contatos/${item._id}`)}>
            <Text>{item.nome}</Text>
            {item.fotoId && (
              <Image source={{ uri: getImageUrl(item.fotoId) }} style={{ width:
100, height: 100, marginVertical: 8 }} />
            )}
            <Button title="Excluir" onPress={() => excluir(item._id)} />
          </TouchableOpacity>
        )}
      />
    </View>
  );
}
```

---

12) Form com upload — `components/FormContato.tsx`

```
import { useState } from 'react';
import { View, TextInput, Button, Image, Alert } from 'react-native';
import * as ImagePicker from 'expo-image-picker';
import api, { getImageUrl } from '../lib/api';
import { global } from '../styles/global';
import { Contato } from '../types/Contato';
```

```
interface Props {
  valores?: Partial<Contato>;
  onSubmit: (dados: Partial<Contato>) => void;
}

export default function FormContato({ valores, onSubmit }: Props) {
  const [nome, setNome] = useState(valores?.nome || '');
  const [email, setEmail] = useState(valores?.email || '');
  const [telefone, setTelefone] = useState(valores?.telefone || '');
  const [endereco, setEndereco] = useState(valores?.endereco || '');
  const [fotoId, setFotoId] = useState<string | undefined>(valores?.fotoId);

  const escolherImagem = async () => {
    const permissoes = await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (permissoes.status !== 'granted') {
      Alert.alert('Permissão necessária', 'Permita acesso à galeria para
escolher uma imagem.');
      return;
    }

    const resultado = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images,
      quality: 0.8
    });

    if (!resultado.canceled && resultado.assets.length > 0) {
      const file = resultado.assets[0];

      const form = new FormData();
      form.append('foto', {
        uri: file.uri,
        name: file.fileName || 'imagem.jpg',
        type: file.mimeType || 'image/jpeg'
      } as any);

      try {
        const { data } = await api.post('/upload', form, {
          headers: { 'Content-Type': 'multipart/form-data' }
        });
        if (!data?.fileId) throw new Error('Upload inválido');
        setFotoId(data.fileId);
      } catch (err: any) {
        Alert.alert('Erro no upload', err?.response?.data?.erro || 'Tente outra
imagem (.jpg/.png/.webp)');
      }
    }
  };

  const enviar = () => {
    if (!nome) { Alert.alert('Validação', 'Nome é obrigatório.'); return; }
    onSubmit({ nome, email, telefone, endereco, fotoId });
  };

  return (
    <View>
      <TextInput placeholder="Nome" value={nome} onChangeText={setNome}
style={global.input} />
      <TextInput placeholder="Email" value={email} onChangeText={setEmail}
style={global.input} autoCapitalize="none" />
      <TextInput placeholder="Telefone" value={telefone}
onChangeText={setTelefone} style={global.input} />
      <TextInput placeholder="Endereço" value={endereco}
onChangeText={setEndereco} style={global.input} />
```

```
      <Button title="Selecionar imagem" onPress={escolherImagem} />
      {fotoId && (
        <Image source={{ uri: getImageUrl(fotoId) }} style={{ width: 120,
height: 120, marginVertical: 8 }} />
      )}

      <Button title="Salvar" onPress={enviar} />
    </View>
  );
}
```

---

13) Novo contato — `app/contatos/novo.tsx`

```
import { View, Alert } from 'react-native';
import { useRouter } from 'expo-router';
import FormContato from '../../components/FormContato';
import api, { loadAuthToken } from '../../lib/api';
import { global } from '../../styles/global';

export default function NovoContato() {
  const router = useRouter();

  const salvar = async (dados: any) => {
    try {
      await loadAuthToken();
      await api.post('/contatos', dados); // { nome, ..., fotoId }
      router.back();
    } catch (err: any) {
      Alert.alert('Erro', err?.response?.data?.mensagem || 'Falha ao criar
contato');
    }
  };

  return (
    <View style={global.container}>
      <FormContato onSubmit={salvar} />
    </View>
  );
}
```

---

14) Editar contato — `app/contatos/[id].tsx`

```
import { useEffect, useState } from 'react';
import { useLocalSearchParams, useRouter } from 'expo-router';
import { View, ActivityIndicator, Alert } from 'react-native';
import FormContato from '../../components/FormContato';
import api, { loadAuthToken } from '../../lib/api';
import { global } from '../../styles/global';
import { Contato } from '../../types/Contato';

export default function EditarContato() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const [contato, setContato] = useState<Contato | null>(null);
  const router = useRouter();

  useEffect(() => {
    (async () => {
      try {
```

```
        await loadAuthToken();
        const { data } = await api.get(`/contatos/${id}`);
        setContato(data);
      } catch {
        Alert.alert('Erro', 'Falha ao carregar contato');
      }
    })();
  }, [id]);

  const atualizar = async (dados: Partial<Contato>) => {
    try {
      await api.put(`/contatos/${id}`, dados);
      router.back();
    } catch (err: any) {
      Alert.alert('Erro', err?.response?.data?.mensagem || 'Falha ao
atualizar');
    }
  };

  if (!contato) return <ActivityIndicator size="large" style={{ marginTop: 50 }}
/>;

  return (
    <View style={global.container}>
      <FormContato valores={contato} onSubmit={atualizar} />
    </View>
  );
}
```

---

16) FAQ — dúvidas que sempre aparecem

1) A imagem não carrega no Android.
Use `10.0.2.2:3000` no emulador. Teste `GET /upload/:id` no próprio dispositivo.

2) Upload falha com 415/400.
Cheque `Content-Type: multipart/form-data` e `name/type` no `FormData`.

3) "Volta para login" depois de logar.
Garanta que o guard (`_layout.tsx`) chama `loadAuthToken()` antes de decidir.

4) Remover foto.
Envie `fotoId: undefined` no PUT e trate no backend apagando o arquivo antigo (se desejado).

5) Cache da imagem mudou?
Use `getImageUrl(fotoId) + '?t=' + Date.now()` após novo upload.

---

15) Teste guiado (roteiro de aula)

1. **Registrar** usuário (erro intencional: deixe o e-mail em branco e mostre a validação).

2. **Login** bem-sucedido (**demonstração**: faça `const t = await loadAuthToken(); console.log('token carregado?', !!t)` para provar que veio do armazenamento seguro).
3. **Criar contato** sem imagem.
4. **Editar contato** e enviar imagem → conferir `fileId` retornado no console da API.
5. **Listar** e ver miniatura renderizando.
6. **Excluir** um contato e observar recarregamento com o listener de `focus`.

---

17) Checklist de compatibilidade com a API v2 (GridFS) de compatibilidade com a API v2 (GridFS)

- Campo `fotoId` no tipo e nas telas.
- Upload em `/upload` com campo `foto` (multipart) e **Authorization**.
- Exibição: `GET /upload/{fotoId}`.
- CRUD: `POST/PUT` enviam `fotoId` quando existir.
- Token JWT persistido com **SecureStore** e aplicado no Axios.

Agora o app usa **SecureStore** para manter o JWT de forma segura, alinhando boas práticas de segurança com a didática em sala de aula.